PROJECT REPORT

SaveIQ

AI-Powered Savings Goal Tracker

Personal Finance • Goal Tracking • AI Financial Coaching • Automated Reminders

Submitted for

NASSCOM Virtual Internship Program – AI Specialist in BFSI

Submitted by

Mohit Mundke

Project Repository

https://github.com/mohitmundke/SavelQ

Deployed Application: https://savel-q.vercel.app/

Abstract

SaveIQ is an AI-assisted personal finance and savings-goal management application. It helps users define financial targets, record deposits, monitor progress, estimate the savings pace required to meet deadlines, and receive AI-generated strategy suggestions. The project combines a browser-based interface with a Google Apps Script and Google Sheets deployment path, plus an optional Python FastAPI service using local database storage and PDF generation. The application also provides data export, printable reporting, preferences, and a savings projection simulator.

1. Introduction

Saving toward a large purchase or financial reserve requires more than setting a target. Users must track contributions, understand the remaining amount, maintain a realistic savings pace, and respond when deadlines approach. SaveIQ brings these activities into one dashboard so users can monitor progress and make informed adjustments. The project focuses on goal-based saving rather than banking transactions, investment execution, or formal financial advice.

2. Problem Statement

People often set savings goals but lack a consistent way to measure progress, calculate how much they should save regularly, or notice when a deadline is at risk. Manual tracking can become fragmented across notes and spreadsheets. SaveIQ addresses this by organizing goals, contributions, deadline calculations, dashboard analytics, AI guidance, and reminder workflows in one application.

3. Objectives

• Create savings goals with a target amount, initial deposit, deadline, category, and optional alert email.

• Track deposits and calculate saved, remaining, and completion amounts.

• Display total target, total saved, funded percentage, and active, completed, and overdue goal counts.

• Provide AI-assisted savings strategies and practical expense-reduction suggestions.

• Support savings projections, data export, printable reports, and deadline audits.

4. Scope and Features

4.1 Savings Goal Management

Users can create, edit, and delete savings goals. The Create Savings Goal form includes a goal name, target amount, initial deposit, target deadline, category, and optional alert email. A goal can be monitored through its saved amount, remaining amount, deadline, category, and progress percentage.

4.2 Dashboard and Goal Status

The dashboard summarizes total saved, target value, funded percentage, discipline score, and the number of active, completed, and overdue goals. Users can filter goals by status, search by goal name, and sort the list by deadline. Individual goal cards display the saved amount, target amount, progress bar, timeline, savings pace, and quick-deposit controls.

The supplied dashboard screenshot displays example values of 9,000 total saved, 11,700 target value, 76.9% funded, a 50% discipline score, and 2,700 remaining. It also shows one active goal, one completed goal, and two overdue goals. These are the values visible in the supplied application screenshot.

4.3 Deposit Contributions

Goal cards provide a Deposit action and quick-save options of 100, 500, 1,000, and 2,500. These controls allow contributions to be recorded against a selected savings goal. Contribution history supports reviewing previous deposits.

4.4 Savings Projection Simulator

The simulator allows a user to select an existing goal or enter a custom target. It accepts target capital, amount already saved, and a weekly contribution rate. The interface displays estimated time to goal, projected date, and daily required savings. It can also explain how increasing the weekly contribution changes the estimated completion time.

4.5 AI Financial Recommendations

SaveIQ connects to Groq Cloud AI for personalized savings plans and expense-trimming suggestions. The repository documentation identifies Llama 3.3 70B Versatile. The supplied Preferences screenshot shows Groq Cloud with GPT-OSS-120B / Llama-3.3 options and an optional OpenAI API key field. Model availability depends on provider configuration.

4.6 Automated Audits, Export and PDF Reporting

The application includes an Audit control for deadline checks. The Google Apps Script backend documented in the repository includes a scheduled audit workflow that checks goals approaching their deadlines or already overdue and can send reminder emails through Gmail services. The export control downloads goal information as a CSV file. The supplied screenshot shows a successful CSV download notification and SaveIQ report files.

The PDF Report control opens a printable statement/report workflow. The supplied print-preview screenshot shows dashboard content prepared for printing to PDF. The optional FastAPI backend documented in the repository also includes a PDF report endpoint implemented with ReportLab.

5. Technology Stack and Architecture

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript	Dashboard, goal forms, projections, filters, and user interactions.
Cloud backend	Google Apps Script	Goal operations, Sheets integration, AI calls, and scheduled audits.
Cloud data	Google Sheets	Stores goal records in the cloud deployment path.
AI service	Groq Cloud API; optional OpenAI key shown in preferences	Generates savings strategies and recommendations.
Email	GmailApp	Sends automated deadline notifications in the Apps Script workflow.
Optional Python	FastAPI, SQLite, ReportLab	REST API, local database operations, and PDF generation.
Export	CSV and browser print-to-PDF	Downloads or prints savings information.

5.1 High-Level Workflow

• The user creates or selects a savings goal in the web interface.

• Goal details are stored through the selected backend and data-storage mode.

• Deposits update saved amount, remaining amount, and progress.

• Dashboard analytics summarize goal-level and portfolio-level information.

6. Savings Calculation Concepts

6.1 Progress Percentage

Progress measures how much of the target amount has been saved. The application uses a capped percentage so that a completed goal does not display progress above 100%.

Progress (%) = min((Saved Amount / Target Amount) × 100, 100)

6.2 Remaining Amount

The remaining amount represents the difference between the target and the amount already saved. The value is not allowed to fall below zero.

Remaining Amount = max(Target Amount − Saved Amount, 0)

6.3 Days Remaining and Savings Pace

The savings pace estimates the contribution required to reach the target by the selected deadline. The application interface displays timeline and daily pace information for individual goals.

Daily Required = Remaining Amount / Days Left
Weekly Required = Remaining Amount / (Days Left / 7)

These calculations apply when the goal has a positive number of days remaining. Overdue and same-day goals require separate status handling.

6.4 Goal Status Classification

Status	Condition	Meaning
Completed	Saved amount ≥ target amount	The target has been fully funded.
Overdue	Deadline passed and target is not yet funded	The deadline has passed while funds remain due.
Active	Target is not funded and deadline has not passed	The goal remains in progress.

6.5 Discipline Score

The dashboard displays a discipline score and descriptive label such as Moderate pacing. In the supplied dashboard screenshot, the score is 50%. This is an application-generated indicator and should not be interpreted as a standardized financial or behavioral assessment.

7. Screenshots of the Application

The following screenshots were supplied by the project developer. They document the SaveIQ dashboard, savings projection simulator, CSV export, print preview, preferences, and goal creation interface. Values shown are the examples visible in the screenshots.

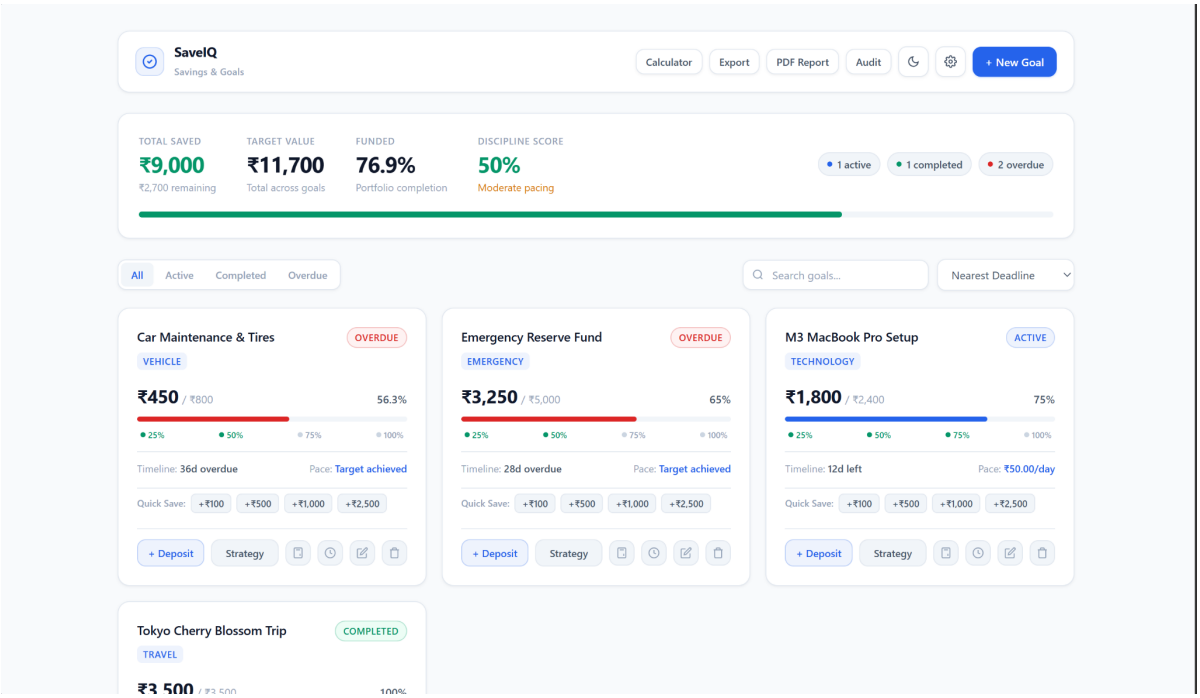


Figure 1. Dashboard showing savings totals, target value, funded percentage, discipline score, status filters, and savings goal cards.

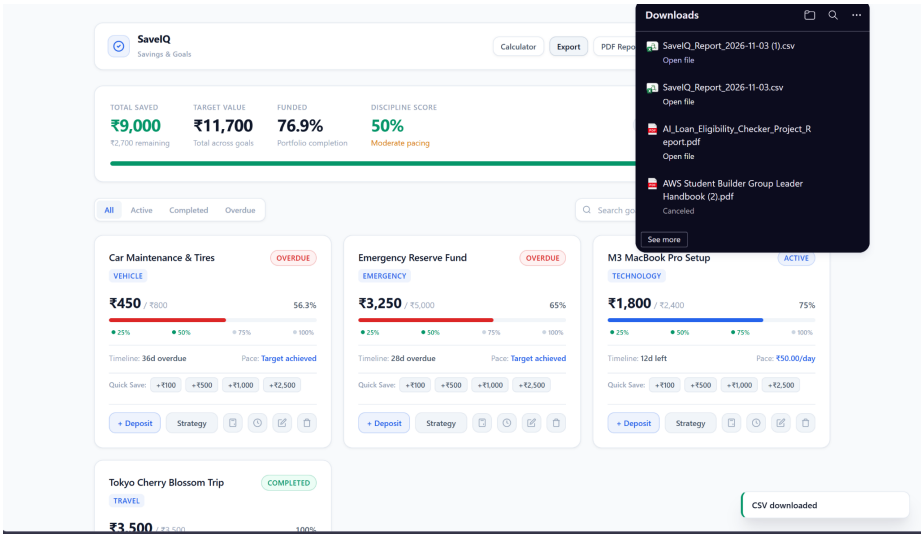


Figure 2. CSV export confirmation and browser download list showing SaveIQ report files.

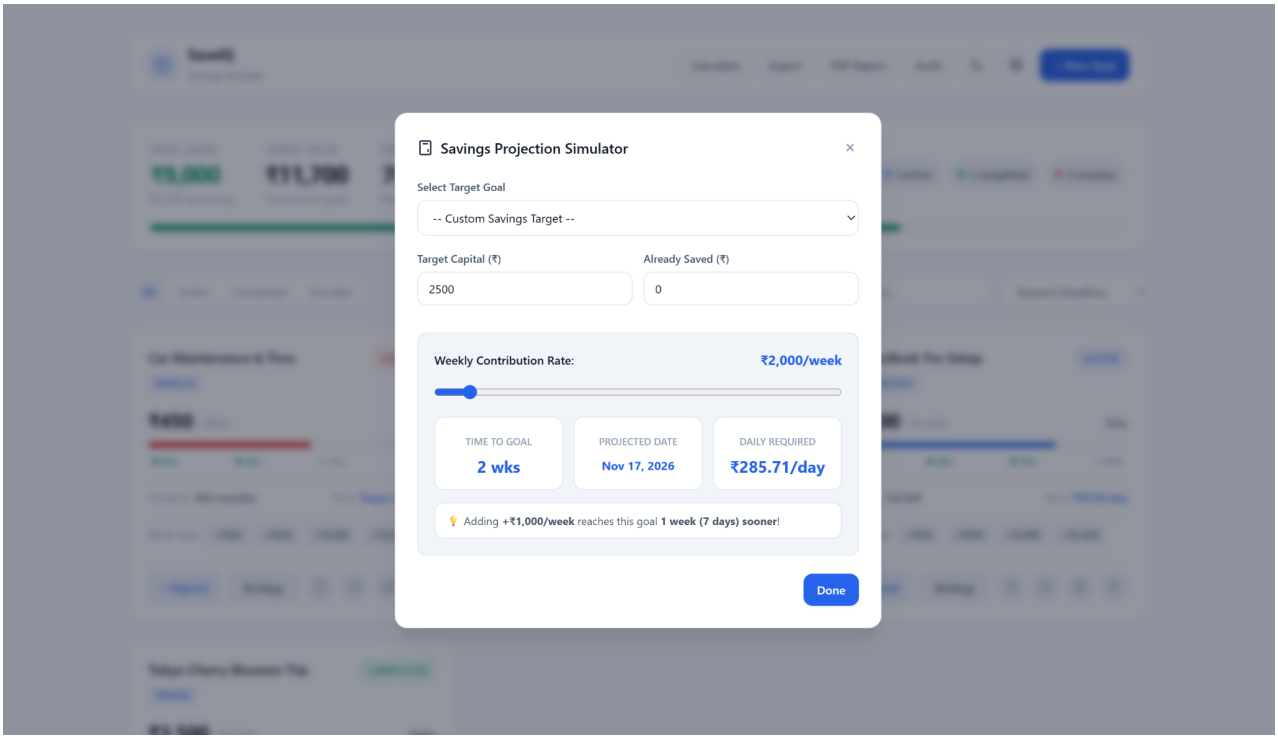


Figure 3. Savings Projection Simulator with custom target capital, already-saved amount, weekly contribution rate, estimated time to goal, projected date, and daily required amount.

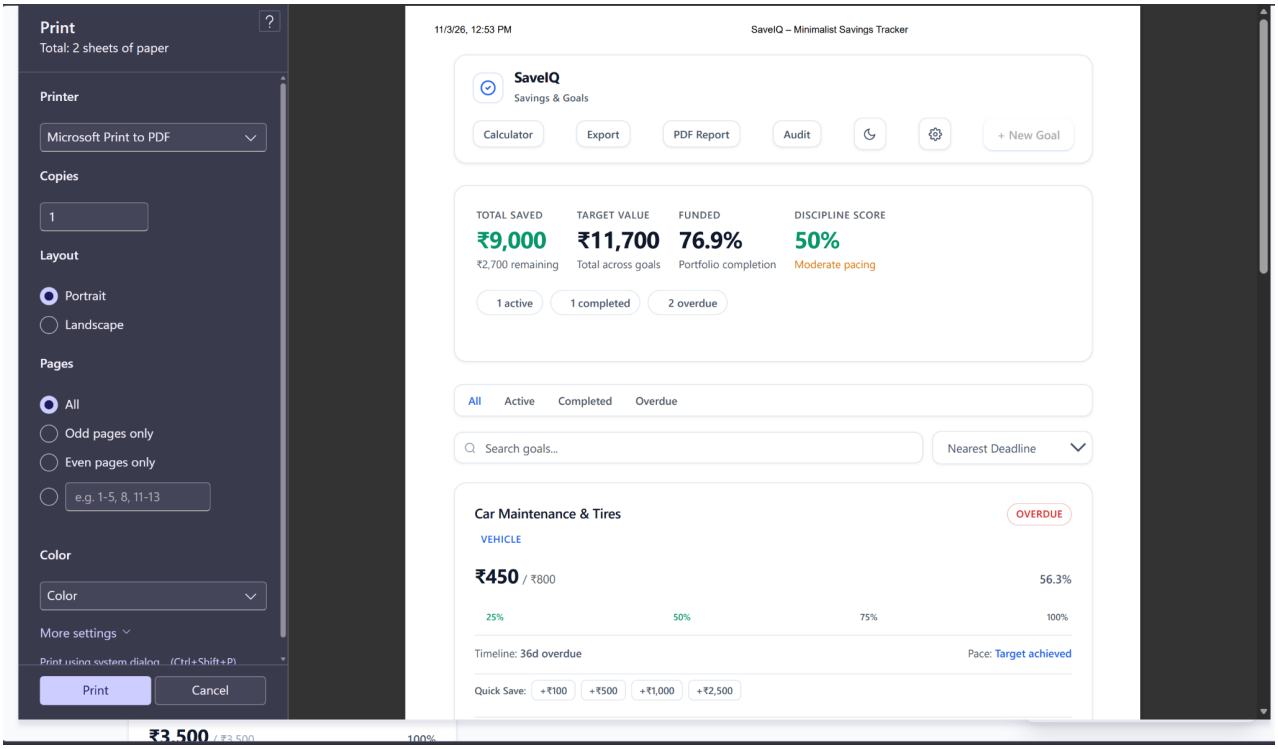


Figure 4. Browser print preview showing the SaveIQ dashboard formatted for printing or saving as a PDF.

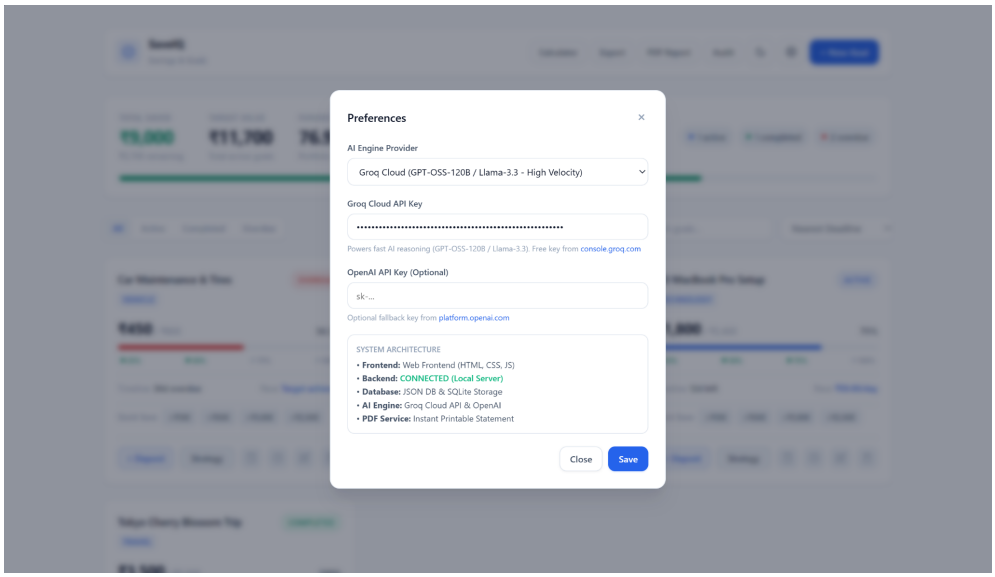


Figure 5. Preferences window showing AI provider selection, masked API key fields, and system architecture information.

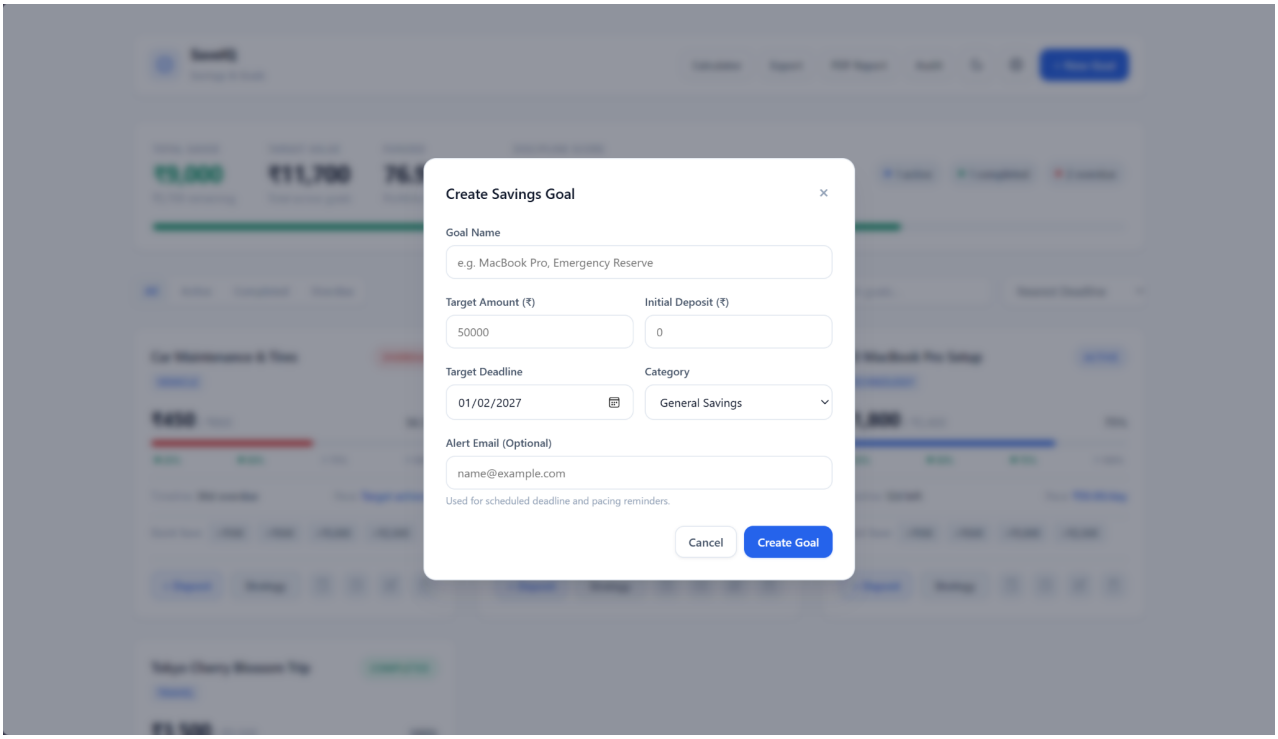


Figure 6. Create Savings Goal form with goal name, target amount, initial deposit, deadline, category, and optional alert email.

8. Security, Privacy and Responsible AI

• Store API keys in server-side environment variables or Google Apps Script PropertiesService. Do not commit secrets to public version control.

• Request only the permissions required for Sheets access, email delivery, and external API requests.

• Validate amounts, dates, email addresses, and identifiers before saving or processing data.

• Avoid sending unnecessary personal or financial information to an AI provider.

• Explain how goal information and email addresses are stored and used, and provide appropriate consent and deletion controls.

• Treat AI-generated recommendations as general educational guidance, not guaranteed financial advice.

• Protect local database files and exported reports containing personal savings information.

9. Testing and Evaluation

The repository includes a Google Apps Script test function named testCompleteWorkflow(). Its documented workflow initializes the database, creates a test goal, checks remaining amount and progress calculations, simulates a deposit, checks dashboard analytics, exercises AI insights or fallback behavior, and cleans up the test goal.

The Python backend exposes REST endpoints for retrieving, creating, updating, and deleting goals; recording deposits; retrieving dashboard statistics; and generating a PDF report. The presence of test code does not by itself prove that every deployment configuration has been tested in production.

10. Limitations

• Savings projections depend on user-entered targets, dates, and contribution assumptions; outcomes are not guaranteed.

• AI recommendations may be incomplete, inaccurate, or unsuitable for an individual's full financial circumstances.

• Apps Script, FastAPI, and browser/local modes have different storage and execution behavior and should not be assumed to operate as one combined runtime.

• Email delivery depends on authorization, configured triggers, valid recipient addresses, and service availability.

• Dashboard figures in the supplied screenshots represent displayed sample application data and may change as goals are updated.

11. Future Enhancements

• Add secure user authentication and clear separation of each user's savings records.

• Provide recurring deposit schedules and optional bank-import integrations with explicit user consent.

• Add charts for monthly contributions, goal comparisons, and historical progress.

• Improve AI recommendations with transparent assumptions and budget-aware scenarios.

• Provide multilingual interface support and accessibility improvements.

• Add configurable reminder frequency, notification preferences, and audit logs.

• Strengthen automated tests across Apps Script, FastAPI, local storage, and deployment environments.

• Add data export, account deletion, and retention controls for personal financial records.

12. Conclusion

SaveIQ is a goal-oriented personal finance application designed to encourage consistent saving through measurable targets, contribution tracking, deadline awareness, dashboard analytics, AI-generated strategies, and automated email reminders. The supplied screenshots demonstrate its dashboard, savings projection simulator, CSV export, printable report workflow, preferences, and goal creation form.

The project brings practical savings-management functions into one interface. Its continued development should focus on reliable testing of financial calculations and scheduled alerts, secure handling of API keys and personal data, clear documentation of deployment modes, and transparent communication that AI suggestions and savings projections are informational rather than guaranteed financial outcomes.

References

• SaveIQ GitHub repository: https://github.com/mohitmundke/SavelQ

• SaveIQ deployed application: https://savel-q.vercel.app/

• Google Apps Script documentation: https://developers.google.com/apps-script

• Google Sheets API documentation: https://developers.google.com/sheets/api

• FastAPI documentation: https://fastapi.tiangolo.com/

• Groq API documentation: https://console.groq.com/docs

• ReportLab documentation: https://docs.reportlab.com/